



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Title of Thesis

Bachelor Thesis

Lars Hulsbergen

Saturday 3rd October, 2026

Advisors: Yakup Koray Budanaz, Lorenzo Paleari

Department of Computer Science, ETH Zürich

Acknowledgments

Abstract

Contents

Contents	iii
1 Introduction	1
1.1 Motivation	1
1.2 Research Questions and Contributions	1
2 Background	2
2.1 HPCAgent-Bench	2
2.2 Compilers, Languages and Code Generation	2
2.3 Iterative Solvers and Preconditioning	2
3 Extracted Application Kernels	3
3.1 Selection	3
3.2 Kernel Descriptions	5
3.3 Design Decisions and Validation	8
4 Loop-Transformation Kernels	10
4.1 Kernel Set	10
4.2 Transformation Classes	11
5 Agent Skills	12
5.1 Kernel-Extraction Skill	12
5.2 Solver-Preconditioning Skill	12
6 Evaluation	13
6.1 Experimental Setup	13
6.2 Language Comparison	14
6.3 Skill Evaluation	18
6.4 Discussion	18
7 Conclusion	19

Bibliography	20
Full Measurement Results	23
Extracted Kernel Catalogue	25
Skill Texts	26
Reproducibility	27

Chapter 1

Introduction

1.1 Motivation

1.2 Research Questions and Contributions

Background

2.1 HPCAgent-Bench

2.2 Compilers, Languages and Code Generation

2.3 Iterative Solvers and Preconditioning

Extracted Application Kernels

Benchmarks for loop optimisation are dominated by short, synthetic loops. They isolate one transformation each, which makes them good diagnostics, but they say little about the code that high-performance applications actually spend their time in. To complement the loop-transformation kernels of Chapter 4, we extracted eight kernels from six production and research codes and added them to HPCAgent-Bench [17]. This chapter describes which kernels we chose and why, what each one computes, and how we established that each port is faithful to its upstream source. The extraction procedure itself, which we also wrote down as an agent skill, is the subject of Chapter 5.

3.1 Selection

We organised the selection by the thirteen computational patterns, or *dwarfs*, of the Berkeley view on parallel computing [1], which HPCAgent-Bench also uses to structure its scientific-computing track. The eight kernels cover six of the thirteen dwarfs (Table 3.2). Within a dwarf, we chose the hot spot of an application that is in active use, and we located it by profiling the application rather than by reading its source. Table 3.1 summarises the profiles. Except for WarpX, which was profiled separately, every application was run on one GH200 node of CSCS Alps daint, with `perf` and DWARF call graphs for the host code and Nsight Systems for the GPU kernels; the raw reports are part of the artifact repository. For five of the six applications the profile points at a single region of code. QuaTrEx is the exception, and we discuss it with its kernel below.

The kernels were chosen to differ from the loop-transformation kernels along three axes. First, *access patterns*: they include a scatter with colliding writes, hash-table insertion, binary search over adjacency lists and a worklist recurrence, none of which is an affine loop nest. Second, *data types*: only the three WarpX kernels compute in `float64`; the others work on 2-bit integer

3.1. Selection

Application	Profiled workload	Hot spot	Share
CoMet	2-way CCC, 24 000 vectors $\times$ 40 000 fields	CUTLASS INT4 GEMM	99.7% GPU
QuaTrEx	carbon-nanotube GW, 100 energies, 4 iterations (CPU backend)	ZGEMM, incl. the RGF block products	$\approx$ 40% CPU
SpBench/cuBool	$C = AA$ on five SuiteSparse graphs	row analysis and binning	55.1% GPU
WarpX	3D plasma, 256^3 cells, 537M particles, 500 steps	hash count and fill field gather (in PushPX)	44.6% GPU $\leq$ 42.7%
		current deposition	40.8%
		Boris push	8.7%
GraphAIBench	triangle counting on com-Orkut	triangle_bs_warp_edge	100% GPU
VASim	Brill, EntityResolution, Fermi automata	per-symbol simulation step	$>$ 99% CPU

Table 3.1: Hot spots of the upstream applications. GPU shares are fractions of GPU kernel time (Nsight Systems), CPU shares fractions of sampled self time (perf).

Kernel	Upstream	Data type
<i>Dense linear algebra</i>		
comet_int4_gemm	CoMet [14, 10]	int4 $\rightarrow$ int32
<i>Sparse linear algebra</i>		
quatrex_rgf	QuaTrEx [7, 21]	complex128
spgemm_hash	SpBench/cuBool [15, 16]	int64 (Boolean)
<i>N-body methods</i>		
warpx_boris_push	WarpX [20]	float64
warpx_esirkepov_deposition	WarpX	float64
warpx_field_gather	WarpX	float64
<i>Graph traversal</i>		
triangle_count	GraphAIBench [5, 6]	int64
<i>Finite-state machines</i>		
nfa_frontier	ANMLZoo/VASim [22, 23]	uint8, int64

Table 3.2: The eight extracted kernels, grouped by Berkeley dwarf.

codes, 64-bit indices, bytes or complex128. Third, *control flow*: several kernels dispatch at run time on the problem configuration or branch on the data itself. The loop-transformation kernels share none of these properties, so the two sets together test whether a statement about code representations holds beyond flat floating-point loops.

3.2 Kernel Descriptions

Each kernel consists of a NumPy reference, which is the correctness oracle, an input generator, a manifest declaring problem sizes and graded outputs, and a native reference kept for provenance. We describe each kernel by what it computes, where the upstream boundary was drawn, and what was deliberately left out.

CoMet INT4 GEMM. CoMet computes pairwise comparison metrics between genomic vectors. Its CCC metric reduces to a tensor-core GEMM over 2-bit field codes: for every pair of vectors (i, j) and every $(e_i, e_j) \in \{0, 1\}^2$,

$$out_{i,j,e_i,e_j} = \sum_f c_{e_i}(L_{i,f}) c_{e_j}(R_{j,f}),$$

where $c_1(v) = \text{popcount}(v)$ and $c_0(v) = 2 - c_1(v)$. The kernel keeps this mathematical contract and drops what has no CPU equivalent: the tensor-core tiling, the MPI runtime, and two adjacent GPU kernels that repack operands and undo a tiling-only row permutation (about 0.1% of GPU time each). The output is accumulated in int32 and graded bit-exactly.

QuaTrEx RGF selected solve. QuaTrEx simulates quantum transport with non-equilibrium Green’s functions. Its core is the recursive Green’s function (RGF) algorithm [18], which computes the block-tridiagonal entries of $X' = A^{-1}$ and of $X^{\lessgtr} = A^{-1} S^{\lessgtr} A^{-H}$ for a block-tridiagonal A without forming the dense inverse. This reduces the cost from $\mathcal{O}((N_B B)^3)$ to $\mathcal{O}(N_B B^3)$ for N_B blocks of size B . The algorithm is a forward and a backward recurrence over the blocks, repeated independently for every energy point; the energy axis is therefore embarrassingly parallel, while the block axis is strictly sequential. We transcribed `RGF.selected_solve` with six simplifications, each recorded in the kernel’s documentation: contact self-energies are pre-folded into the supplied blocks, both triangles of the output are written, the current is not computed, energy batching (a memory-allocation strategy) is dropped, block sizes are uniform, and the batched inverse becomes one inverse per block.

Unlike the other applications, QuaTrEx has no single hot spot. With the CPU backend, about 40% of a carbon-nanotube GW run is spent in complex matrix multiplication (ZGEMM), the operation the RGF recurrence consists of. The profile resolves only native frames, however, so it cannot attribute these products to the RGF solve rather than to other matrix products in the GW loop. The eigensolve of the open-boundary conditions and the FFT-based GW convolution take about 9% each. With the GPU backend, no kernel exceeds 8.6% of GPU time among about 280 000 launches, and the host side is dominated by the same eigensolve. We extracted the RGF solve because it is the algorithmic core of the transport calculation, not because the profile isolates it.

Boolean SpGEMM with hash accumulation. The kernel computes $C = AB$ over the Boolean $(\vee, \wedge)$ semiring for matrices in CSR format, the operation behind cuBool [16], which uses the nsparse algorithm [13]. Row i of C is the union of the rows of B selected by row i of A , so the cost lies in de-duplicating that union, not in arithmetic. Upstream computes it in five phases: an upper bound on each row's size, binning of rows by that bound, a symbolic phase that counts distinct columns in a per-row hash table, a scan, and a numeric phase that refills the hash tables and sorts each with a bitonic network. Profiling cuBool on five SuiteSparse graphs attributes 55.1% of GPU time to row analysis and binning and 44.6% to the two hash phases, so we ported all five phases rather than only the hashing. We left out the path for rows with more than 4096 products. The segmented sort it ends in takes 0.1% of GPU time, and the path is also incorrect: it appends products without removing duplicates, so on the web-Google graph the one row that takes it holds 3819 duplicate column indices.

WarpX profile. WarpX is a particle-in-cell code for plasma and accelerator physics. To decide which of its kernels to extract, we profiled a 3D uniform-plasma run with perf, using inputs scaled from the WarpX performance study of Myers et al. [12]: a 256^3 -cell grid over a $40\text{ }\mu\text{m}$ cube, electrons and protons with eight particles per cell each (about 537 million particles), cubic particle shapes, Esirkepov current deposition, the Boris pusher, the Yee field solver and periodic boundaries, run for 500 time steps without I/O. The run took 16 min 20 s; Table 3.1 shows where the time went. The field gather has no entry of its own: it is inlined into the particle-push loop PushPX, whose self time of 42.7% contains the gather together with the rest of that loop, so the profile bounds the gather's share from above rather than isolating it. Current deposition takes another 40.8% and the Boris update 8.7%, while the Yee field solver takes only 0.5%. Together, the particle kernels and the loop around them account for about 92% of the run, so we extracted all three.

WarpX Boris push. The Boris pusher [3] advances every particle's momentum by one relativistic rotation in the local electromagnetic field. The port preserves all three push variants (full, first half, second half), including the rescaling that makes two half pushes equal one full push. Each particle reads and writes only its own elements, so the kernel is a pure elementwise map and its outputs are graded bit-exactly.

WarpX Esirkepov current deposition. The Esirkepov scheme [8] deposits each particle's current onto the grid such that the discrete charge-continuity equation holds exactly. The port keeps every branch of the upstream kernel: all six geometries, shape orders one to four, the reduced-shape deposition near embedded boundaries, ionisation-level weighting, and the azimuthal-

mode terms of the cylindrical geometry. Unlike the other two WarpX kernels, deposition is a *scatter*: neighbouring particles write to the same grid cells. Upstream resolves the collisions with atomic additions, which reorder the floating-point accumulation from run to run. The benchmark grades bitwise reproducibility across two runs, so a correct parallel version must fix the summation order, for example with per-thread grids reduced in a fixed order.

WarpX field gather. The field gather is the inverse operation: it interpolates the electric and magnetic fields from the staggered grid onto every particle with the same shape functions. The grid is only read and each particle writes only its own six outputs, so the particle loop is embarrassingly parallel and bit-exact under any schedule. The kernel keeps all geometries and shape orders; Section 3.3 explains why the graded run nevertheless fixes the geometry.

Triangle counting. The kernel counts the triangles of an undirected graph with the edge-parallel, binary-search variant of GraphAIBench [5]. The graph is first oriented into a directed acyclic graph by degree, so that each triangle is counted exactly once; the count is then the sum, over all directed edges (u, v) , of the sizes of the intersections of the out-neighbour lists of u and v . Each intersection searches the shorter list in the longer one with a two-phase binary search: 32 evenly spaced samples first bracket the key, and a second search finishes within the bracket. This is what lets the kernel handle adjacency lists whose lengths differ by orders of magnitude. The edge loop is a parallel reduction into a single integer. On SNAP’s com-Orkut graph, this kernel is the application’s only GPU kernel; it counts the published 627 584 181 triangles in 68.3 ms.

NFA frontier simulation. The kernel simulates a homogeneous nondeterministic finite automaton over a byte stream, the inner loop of automata processing as defined by the ANMLZoo suite [22] and its reference engine VASim [23]. Each cycle matches the current symbol against the enabled states, enables the successors of the matched states, and re-enables the start states. An automaton is a union of disconnected components, one per pattern, and VASim parallelises over these components; the kernel keeps that decomposition as its outer loop. Inside a component, consecutive cycles form a strict recurrence. On the Brill, EntityResolution and Fermi automata, this per-symbol step takes over 99% of the run, most of it in enabling successors (48–69%) and matching symbols (27–34%). We omitted ANML counter and gate elements: of the twelve ANMLZoo benchmarks only Snort uses them, where they make up 1% of the elements. In Snort’s profile, the time they cost is dominated by container overhead rather than automaton logic: traversing a `std::map` and comparing and copying its string keys (`_Rb_tree_increment`

12.1%, memcmp 10.7%, memcpy 7.9%). Extracting them would benchmark that container choice.

3.3 Design Decisions and Validation

Native references. The native reference beside each kernel takes one of three forms. For SpGEMM, triangle counting and the NFA, it is the upstream source itself, excerpted verbatim. For the three WarpX kernels and CoMet, the upstream code depends on AMReX or CUTLASS and cannot be compiled on its own; there it is a standalone C++ transcription that keeps the upstream operation order and replaces the GPU launch with an OpenMP loop. For QuaTrEx, which is written in Python, it is a NumPy transcription that keeps the upstream expressions verbatim. Two of the references, SpGEMM and triangle counting, are CUDA and cannot be executed on a CPU; this matters for the language comparison in Chapter 6.

Problem sizes. Every kernel declares four size presets, S, M, L and XL, where XL is sized above 4 GB by benchmark convention. Where possible, we anchored a preset to a real input. The M preset of the NFA kernel holds 42 273 states in 1281 components over a 1 MB stream, close to ANMLZoo’s Brill automaton (42 658 states in 1962 components), and its L preset approaches the size of the EntityResolution automaton. The L preset of QuaTrEx uses block size 32, the block size of a QuaTrEx carbon-nanotube simulation.

Pinned configuration. Some kernels have configuration inputs besides their sizes, such as the geometry of the WarpX kernels. A configuration input is either swept as an axis or pinned to a compile-time constant for every framework. For the field gather we pinned the geometry (3D), the interpolation mode and the number of azimuthal modes. As run-time inputs, they kept every branch of the six-geometry, five-shape-order dispatch alive in the dataflow graph, and the DaCe backend exceeded the 1200 s limit of the benchmark’s numerical-oracle test. Pinning reduced the emitted C++ code from 36 251 to 29 339 lines. The shape order cannot be pinned, because it also determines array sizes, so the five-way dispatch over it remains. The other geometries are still tested against the native reference, outside the graded run.

Validation. Each port was validated in layers. The benchmark’s own tests check that every generated backend reproduces the NumPy reference, and a kernel-local test checks the NumPy reference against the native one. Beyond that, we validated each kernel against something that does not share its implementation:

- **QuaTrEx:** bit-for-bit agreement with the upstream transcription, plus the physical symmetries of the output (anti-Hermitian diagonal blocks, $X_{ji} = -X_{ij}^H$). A port that drops a complex conjugate in the same way as its reference would pass the first check but fail the second.
- **SpGEMM:** a set-union oracle that owes nothing to hash tables or bins, a check that the rows are actually spread over several bins, and a comparison with the running upstream library on SuiteSparse road networks.
- **Triangle counting:** complete graphs K_n , whose $\binom{n}{3}$ triangles are known in closed form. The sizes $n = 12$ and $n = 40$ were chosen to exercise the edge case in which an adjacency list is shorter than the 32 samples.
- **NFA:** an independent formulation as a dense Boolean matrix-vector product over the whole automaton, plus a comparison with VASim's activation histogram, which agrees state for state on four ANMLZoo automata (Levenshtein, Brill, Snort and Fermi). Report counts agree on all of them except Snort, where the omitted counter and gate elements account for the difference.
- **WarpX:** the compiled C++ transcriptions against the NumPy port across each kernel's full configuration space (every geometry, shape order and, for the cylindrical geometry, several azimuthal mode counts), built without FMA contraction so that the operation order matches NumPy.
- **CoMet:** the C++ extraction against the NumPy port on deterministic, random and edge-case inputs, and at several thread counts.

Every simplification relative to upstream is documented in the kernel's source, since an undocumented simplification cannot be told apart from a bug.

Loop-Transformation Kernels

The second kernel set isolates the loop transformations that the extracted kernels of Chapter 3 mix with everything else. It consists of 40 kernels of HPCAgent-Bench whose efficient execution depends on dependence analysis and a loop transformation, such as interchange, distribution or fusion, rather than on a library call. We refer to it as the loop-transformation set, or LLR-40 after the benchmark’s tag for it. This chapter describes the kernels and the classes we group them into; Chapter 6 measures them.

4.1 Kernel Set

Twenty-nine of the 40 kernels come from TSVC-2 [19], the C version of the classic test suite for vectorising compilers [4, 11]: 26 keep their TSVC names, and three are renamed variants (`argmax_with_index` is TSVC `s315`, `ext_break_capture` is `s332`, `ext_war_unit` is `s121`). The remaining eleven were written for the benchmark and cover patterns TSVC lacks: producer and consumer loops that must be fused through a temporary array, scans and ragged reductions, a scatter with duplicate indices, stream compaction, a dependence whose distance is known only at run time, and two wavefronts.

Every kernel is written once as a NumPy reference made of explicit loops, which is the correctness oracle. From it, the benchmark’s translators emit C, C++, Fortran and Numba versions. For 35 of the 40 kernels the benchmark also carries a hand-written C reference ported from TSVC-2, kept deliberately separate from the translated C so that the two can be compared. All kernels compute in `float64`. At the problem size we measure (preset M), the translated C version of a kernel takes between 39 ms and 789 ms per call, with a median of 100 ms, which is long enough that loop start-up and timer resolution do not matter.

Class	n	Kernels
Reduction	10	s311, s3110, s3111, s3112, s316, s318, s319, argmax_with_index, quasi_affine_reduce_odd, scan_affine_decay
Loop interchange	5	s1232, s2233, s231, s233, s235
Control flow	5	s2710, s275, vpvts, vtvvtv, ext_break_capture
Indirect addressing	4	s4112, vag, scatter_accum_dup, segment_reduce_ragged
Loop fusion	3	fuse_diamond, fuse_move_ifs, fuse_stencil_through_transient
Dependence distance	2	ext_war_unit, versioned_distance_update
Linear dependence	2	s115, s119
Scalar expansion	2	s252, s255
Wavefront	2	wf_diff_skew, wf_triangular
Loop distribution	1	s2275
Node splitting	1	s1244
Interprocedural data flow	1	s152
Recurrence	1	s323
Packing	1	compact_threshold_pack

Table 4.1: The 40 loop-transformation kernels by class. TSVC kernels are named by their TSVC identifier (s311 is `tsvc.2.s311` in the benchmark).

4.2 Transformation Classes

We group the kernels by the transformation they are meant to exercise (Table 4.1). For 31 kernels the class is declared in the benchmark manifest; for the other nine, whose manifests declare none, we derived it from the loop body. A hand check of the nine derived labels against the kernel sources agrees with six, finds two disputable, and disputes one. The two disputable ones are `ext_break_capture`, which the benchmark files as a search loop in its TSVC original, and `ext_war_unit`, filed under induction variables. The disputed one is `wf_diff_skew`, whose dependences are all carried by the outer loop, so its inner loop is already parallel and needs no wavefront transformation. Two of the declared labels are also questionable: `vpvts` and `vtvvtv` are filed as control flow but contain no branch. We keep all labels as recorded, so that the measurements remain comparable with the benchmark’s own, and point out where a label matters.

The classes are uneven. Reduction is the largest with ten kernels, and nine of the fourteen classes have at most two. A per-class average over one or two kernels is a case study, not an estimate, so the evaluation reports per-kernel results and uses the classes only to organise them.

Agent Skills

5.1 Kernel-Extraction Skill

5.2 Solver-Preconditioning Skill

Evaluation

6.1 Experimental Setup

Platform. All measurements ran on CSCS Alps daint, whose nodes carry four NVIDIA GH200 modules with 72 Arm Neoverse V2 cores each. Every kernel ran on its own exclusive node, and the timed code used a single core, so the measurements compare the quality of serial code, not the ability to parallelise.

Toolchain and flags. The benchmark harness compiles C with GCC 14.2.0, because its C flags select the C23 standard, which GCC 13 does not accept. It compiles C++ with g++ 13.3.1 and Fortran with gfortran 13.3.1. All three receive `-O3 -march=native`, the floating-point relaxations `-fno-math-errno -fno-trapping-math -fno-signed-zeros`, and `-ffp-contract=fast`, which allows multiply and add to be fused into one FMA instruction. `-ffast-math` and `-fassociative-math` are not used: they change the values a kernel computes and would fail the elementwise comparison against the NumPy oracle. Numba 0.67 compiles through LLVM with `parallel=True`, and never with `fastmath`. Thread counts are pinned to one for OpenMP and Numba alike.

Representations. Each kernel is measured in six versions, which we call columns: the C, C++, Fortran and Numba code emitted by the benchmark’s translators from the NumPy reference; the hand-written C reference where one exists; and an agent-optimised version. The agent column takes, for each kernel, the submission that ranked highest in the benchmark’s LLR-40 agent campaigns and still validates on our machine. The submissions come from three open-weight models (Qwen3.8-27B, Kimi-K2.7 and GPT-OSS-120B), are written in C or Fortran, and were produced and graded on an AMD MI300A

node. They are used exactly as delivered: OpenMP-parallel code, run here on one thread. NumPy is the correctness oracle and is not timed.

Protocol. The measurement protocol was fixed before the first measurement and not changed afterwards. Every cell is validated against the NumPy oracle at the same problem size (preset M) and in `float64` before it is timed. It is then run five times as warm-up and 30 times timed, and every individual timing is stored. A cell that fails is recorded with its status (*unsupported* when the version does not exist, *incorrect* when its output does not match the oracle), never dropped.

Statistics. About 14% of all runs are slowed by roughly $1.6\times$ by an intermittent loss of memory bandwidth on the nodes. We excluded clock frequency, core migration, huge-page fallback and NUMA page migration by direct measurement; the hardware counters show the core waiting on DRAM at a constant clock, so the cause lies outside the measured process. It leaves 54 of the 229 valid cells with a relative standard deviation above 5%, while the median cell varies by 1.2%. We therefore report every cell by the minimum of its 30 timings, which this artefact cannot lower. A speedup is a ratio of two such minima. Following Hoeﬂer and Belli [9], a summary over kernels is the geometric mean of the per-kernel ratios with its 95% Student-*t* interval in log space.

Reproducibility. We measured the loop-transformation matrix twice, more than three weeks apart, at two commits of the benchmark whose translators had changed in between. The statuses of all 240 cells are identical, and 214 cells agree within 5%. All 26 cells that moved run code that the compiler sees unchanged, and 19 of them have noisy series. We report the second measurement.

6.2 Language Comparison

Of the 240 cells of the loop-transformation matrix, 229 are valid. Six are unsupported: five kernels have no hand-written C reference, and one kernel received no agent submission. Five are incorrect, all of them `s115`, for a reason the last paragraph of this section explains. Figure 6.1 shows every valid cell as a speedup over the translated C version of the same kernel, and Table 6.1 summarises each column. The full matrix is in Chapter .

The language matters little on a shared backend. Translated into C, C++ and Fortran and compiled by GCC, the same kernel runs at nearly the same speed. Neither `g++` ($1.04\times$ the speed of translated C) nor `gfortran` ($0.98\times$) differs from C beyond its 95% interval, and the three agree within 5% on

6.2. Language Comparison

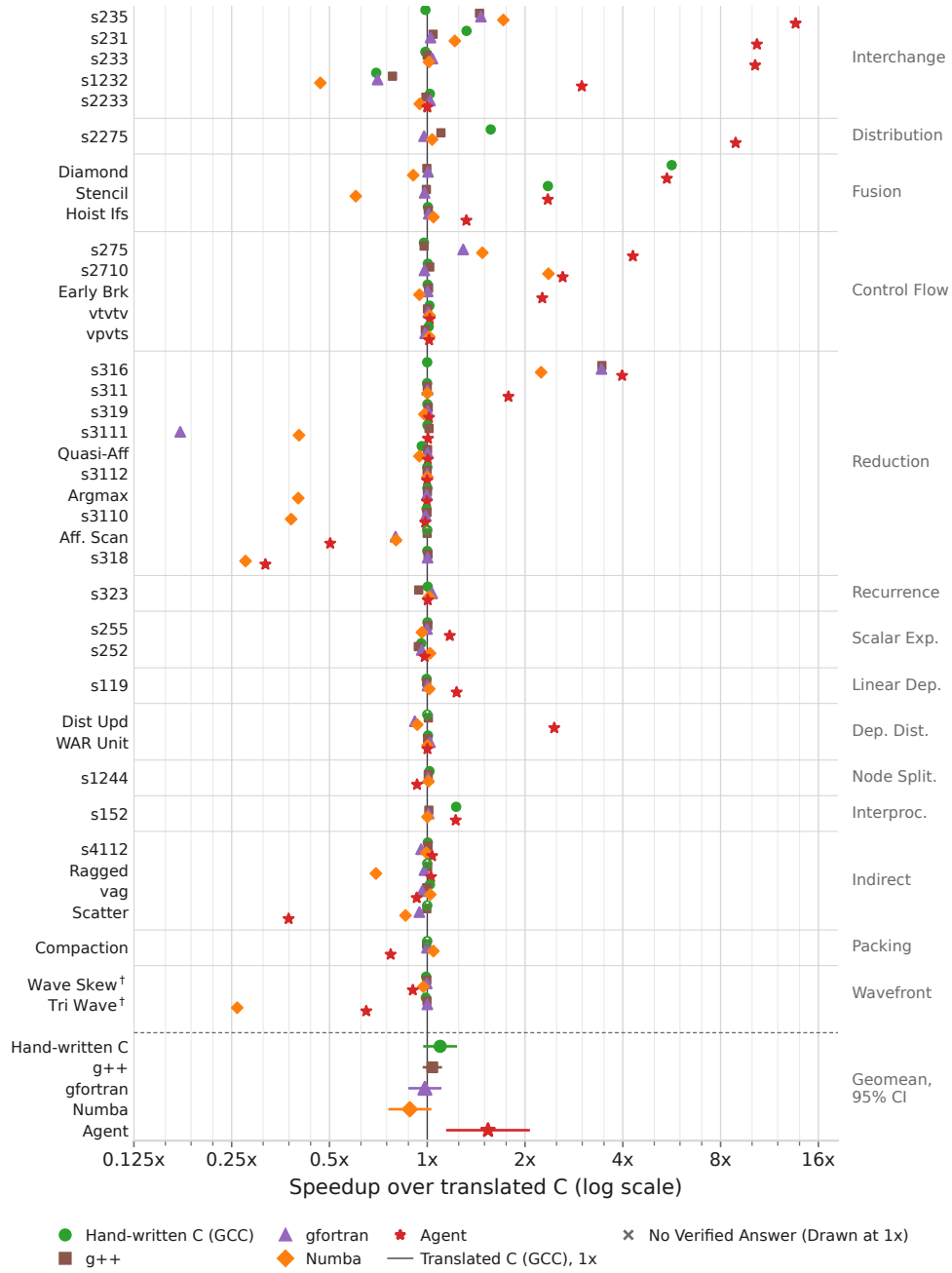


Figure 6.1: Speedup of each version over the translated C version of the same kernel, at preset M on one core of a GH200 node (minimum of 30 runs). Kernels are grouped by transformation class; the bottom rows give each column's geometric mean with its 95% interval. A hollow cross at 1x marks a missing version. s115 is not shown, because no GCC version produces a valid result at this size. [†]The two wavefront kernels pass validation only vacuously at preset M, as explained at the end of this section.

6.2. Language Comparison

Column	n	Speedup over C (95% CI)	Faster	Tie	Slower
Hand-written C (GCC)	34	$1.10 \times [0.97, 1.24]$	5	26	3
g++	39	$1.04 \times [0.97, 1.11]$	4	32	3
gfortran	39	$0.98 \times [0.87, 1.11]$	5	26	8
Numba	39	$0.88 \times [0.76, 1.03]$	8	15	16
Agent	38	$1.54 \times [1.14, 2.07]$	18	12	8

Table 6.1: Geometric-mean speedup of each column over translated C, over the kernels where both are valid, with the number of kernels on which the column is more than 3% faster, within 3%, or more than 3% slower.

27 of 39 kernels. No language wins consistently: among the four translated versions, Numba is fastest on 13 kernels, C++ on 12, Fortran on 8 and C on 6.

The large gaps are individual code-generation decisions. Where the languages do differ, each gap traces to one decision in the generated code, not to the language. On `s3111`, a conditional sum, `gfortran` is $5.8 \times$ slower than C. The translator writes every accumulation in parentheses, `sum = (sum + a(i))`, and in Fortran parentheses are binding, so `gfortran` keeps them and does not recognise the reduction. Removing only the parentheses makes it vectorise the loop. The same parentheses prevent an FMA in the recurrence of `scan_affine_decay`, where Fortran is $1.25 \times$ slower. On `s316`, a running minimum, it is C that is $3.4 \times$ slower than C++ and Fortran, although the C and C++ sources are identical. GCC 14, which compiles the C columns, turns the update into a branch-free select that puts the comparison on the loop-carried dependence chain, while GCC 13 keeps a branch that almost never mispredicts on random data. Rebuilding the same sources with the other compiler version swaps the behaviour, so the difference is the compiler version, not the language. The protocol had expected the opposite effect on reductions: that `gfortran`’s licence to reassociate would make Fortran faster. In the matrix, `gfortran` vectorises none of the ten reductions.

Numba’s results are LLVM’s. Numba is the most uneven column: faster than C on 8 kernels, slower on 16, and $0.88 \times$ the speed of C overall. It is $2.4 \times$ faster than C on `s2710` and $1.5 \times$ faster on `s275`, two kernels with branches in the loop, which LLVM vectorises and GCC does not. It is 2.5 to $3.6 \times$ slower on the four reductions whose update is conditional (`s318`, `s3110`, `argmax_with_index`, `s3111`): LLVM turns the conditional update into the same branch-free select that slows the C version of `s316`, while GCC keeps a branch or, on `s3111`, vectorises. Whether a compiler implements a loop-carried comparison as a branch or as a select is the single most frequent cause of a large gap in the matrix, and it decides kernels in both directions.

No compiler restructures; the agents do. In the five interchange kernels and the distribution kernel, GCC's optimisation reports contain no interchange and no distribution in any column. For `s231`, where the interchange is legal, GCC's interchange pass did not even take up the loop nest as a candidate. The agents interchanged, distributed or flattened these loops by hand, after which GCC vectorised the unit-stride inner loop they had produced; this makes the agent version 3.0 to $9.9\times$ faster than the fastest other version. The hand-written C column shows the same effect from the other side. It ties translated C on 26 of 34 kernels, but is $5.6\times$ faster on `fuse_diamond` and $2.3\times$ faster on `fuse_stencil_through_transient`. There the hand-written reference is the original, already fused TSVC loop, while the NumPy reference, and hence every translated version, is deliberately unfused, and GCC does not fuse it. On these kernels the hand-written column is the optimised answer, not a baseline.

Agent code carries its target machine. The agent column is the fastest overall ($1.54\times$ the speed of C), but slower than C on eight kernels. In the submissions we inspected, the cause is parallel scaffolding that costs work when it runs on one thread. The submission for `scan_affine_decay` is a three-pass blocked parallel scan that reads its input twice, and is $2.0\times$ slower than the sequential recurrence. The one for `scatter_accum_dup` hard-codes the MI300A node it was written on: 48 threads, 24 cores per rank, pinning to SMT siblings, and atomic compare-and-swap updates. It overrides the single-thread setting and is $2.7\times$ slower. Portability fails more directly elsewhere: for 12 of the 13 kernels where we could not use the campaign's best submission, that submission included x86 intrinsics headers and does not compile on an Arm machine. A speedup an agent achieved on its own machine does not by itself transfer to another one.

Vectorising is not the same as being fast. According to its optimisation reports, GCC vectorises a loop in 15 of 39 kernels in the C column, 12 in the Fortran column and 27 in the agent column; Numba's assembly contains vector loops for 19 kernels. Vectorising alone does not predict speed. On the plain sum `s311`, GCC vectorises the C version as an in-order SVE reduction, yet it runs exactly as fast as the scalar Fortran version (135 ms), because an in-order reduction is still bound by the latency of its additions. Only the agent's OpenMP reduction, which permits reassociation, is faster (76 ms). This agrees with what Bonsall and Budanaz [2] observed for code generated by DaCe.

Floating-point contraction decides correctness. `s115` accumulates values that span hundreds of orders of magnitude, so at preset M its output is almost entirely infinite or NaN. With `-ffp-contract=fast`, GCC fuses its

multiply-subtract into an FMA, which rounds differently from NumPy's separate operations and changes where the overflow occurs, so all four GCC versions fail validation. With `-ffp-contract=off`, and nothing else changed, all four pass. Numba passes in both cases because it never enables `fastmath` and reproduces NumPy's arithmetic exactly. The two wavefront kernels overflow as well, but consist only of additions, so every version produces the same mostly non-finite output and passes vacuously. At the largest sizes whose output is fully finite (118 for `s115`, 517 and 1017 for the wavefronts), every version passes, and the wavefront timings show the same pattern as at preset M: Numba $3.6\times$ and the agent $1.6\times$ slower than C on `wf_triangular`.

6.3 Skill Evaluation

6.4 Discussion

Chapter 7

Conclusion

Bibliography

- [1] Krste Asanović, Rastislav Bodik, Bryan Catanzaro, Joseph Gebis, Parry Husbands, Kurt Keutzer, David Patterson, William Plishker, John Shalf, Samuel Williams, and Katherine Yelick. The landscape of parallel computing research: A view from Berkeley. Technical Report UCB/EECS-2006-183, EECS Department, University of California, Berkeley, 2006.
- [2] Alexander Vali Bonsall and Yakup Koray Budanaz. Evaluating the auto-vectorization amenability of DaCe-generated code, 2026. Semester project, ETH Zurich.
- [3] Jay P. Boris. Relativistic plasma simulation – optimization of a hybrid code. In *Proceedings of the Fourth Conference on Numerical Simulation of Plasmas*, pages 3–67. Naval Research Laboratory, 1970.
- [4] David Callahan, Jack Dongarra, and David Levine. Vectorizing compilers: A test suite and results. In *Proceedings of the 1988 ACM/IEEE Conference on Supercomputing*, pages 98–105, 1988.
- [5] Xuhao Chen. GraphAIBench: A benchmark suite for graph machine learning. <https://github.com/chenxuhao/GraphAIBench>.
- [6] Xuhao Chen, Roshan Dathathri, Gurbinder Gill, and Keshav Pingali. Pangolin: An efficient and flexible graph mining system on CPU and GPU. *Proceedings of the VLDB Endowment*, 13(8), 2020.
- [7] Computational Nanoelectronics Group, ETH Zurich. QuaTrEx. <https://github.com/quatrex/quatrex>.
- [8] T. Zh. Esirkepov. Exact charge conservation scheme for particle-in-cell simulation with an arbitrary form-factor. *Computer Physics Communications*, 135(2):144–153, 2001.

- [9] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*, pages 73:1–73:12, 2015.
- [10] Wayne Joubert, James Nance, Sharlee Climer, Deborah Weighill, and Daniel Jacobson. Parallel accelerated custom correlation coefficient calculations for genomics applications. *Parallel Computing*, 84:15–23, 2019.
- [11] Saeed Maleki, Yaoqing Gao, María J. Garzarán, Tommy Wong, and David A. Padua. An evaluation of vectorizing compilers. In *Proceedings of the 2011 International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pages 372–382, 2011.
- [12] A. Myers, A. Almgren, L. D. Amorim, J. Bell, L. Fedeli, L. Ge, K. Gott, D. P. Grote, M. Hogan, A. Huebl, R. Jambunathan, R. Lehe, C. Ng, M. Rowan, O. Shapoval, M. Thévenet, J.-L. Vay, H. Vincenti, E. Yang, N. Zaïm, W. Zhang, Y. Zhao, and E. Zoni. Porting WarpX to GPU-accelerated platforms. *Parallel Computing*, 108:102833, 2021.
- [13] Yusuke Nagasaka, Akira Nukada, and Satoshi Matsuoka. High-performance and memory-saving sparse general matrix-matrix multiplication for NVIDIA Pascal GPU. In *46th International Conference on Parallel Processing (ICPP)*, pages 101–110, 2017.
- [14] Oak Ridge National Laboratory. CoMet. <https://github.com/wdj/comet>.
- [15] Egor Orachyov. SpBench: Sparse boolean linear algebra libraries benchmark. <https://github.com/EgorOrachyov/SpBench>.
- [16] Egor Orachyov, Pavel Alimov, and Semyon Grigorev. cuBool: Sparse Boolean linear algebra for Nvidia Cuda. <https://github.com/JetBrains-Research/cuBool>, 2021. Version 1.2.0.
- [17] SPCL, ETH Zurich. HPCAgent-Bench. <https://github.com/spcl/HPCAgent-Bench>.
- [18] A. Svizhenko, M. P. Anantram, T. R. Govindan, B. Biegel, and R. Venugopal. Two-dimensional quantum mechanical modeling of nanotransistors. *Journal of Applied Physics*, 91(4):2343–2354, 2002.
- [19] UoB-HPC. TSV_C_2: An updated c version of the test suite for vectorising compilers. https://github.com/UoB-HPC/TSVC_2.

- [20] Jean-Luc Vay et al. WarpX: An advanced particle-in-cell code. <https://blast-warpx.github.io>, 2018.
- [21] Nicolas Vetsch, Alexander Maeder, Vincent Maillou, Anders Winka, Jiang Cao, Grzegorz Kwasniewski, Leonard Deuschle, Torsten Hoefler, Alexandros Nikolaos Ziogas, and Mathieu Luisier. Ab-initio quantum transport with the GW approximation, 42,240 atoms, and sustained exascale performance. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*, pages 1–13, 2025.
- [22] Jack Wadden, Vinh Dang, Nathan Brunelle, et al. ANMLZoo: A benchmark suite for exploring bottlenecks in automata processing engines and architectures. In *2016 IEEE International Symposium on Workload Characterization (IISWC)*, pages 1–12, 2016.
- [23] Jack Wadden and Kevin Skadron. VASim: An open virtual automata simulator for automata processing application and architecture research. Technical Report CS2016-03, University of Virginia, 2016.

Full Measurement Results

Table 1 lists every cell of the loop-transformation matrix as the minimum of its 30 timed runs, in milliseconds, measured as described in Section 6.1. The fastest valid version of each kernel is set in bold.

Kernel	C	Hand-written C	C++	Fortran	Numba	Agent
<i>Interchange</i>						
tsvc.2.s1232	39.1	56.2	50.1	55.6	83.5	13.1
tsvc.2.s2233	430.4	422.6	434.6	422.0	453.9	—
tsvc.2.s231	723.3	547.5	694.0	706.7	594.7	69.9
tsvc.2.s233	681.6	690.9	681.9	657.1	673.5	66.7
tsvc.2.s235	789.2	799.2	545.0	539.4	460.2	58.0
<i>Distribution</i>						
tsvc.2.s2275	668.7	426.5	606.9	684.1	646.0	75.1
<i>Fusion</i>						
fuse.diamond	168.8	29.8	169.3	167.8	186.4	30.9
fuse.move_ifs	68.2	67.9	67.7	67.5	65.3	51.7
fuse.stencil_through_transient	112.3	47.7	113.0	114.4	186.3	47.8
<i>Control Flow</i>						
ext.break_capture	67.1	66.9	66.4	66.9	70.9	29.7
tsvc.2.s2710	220.6	219.7	216.7	225.1	93.4	84.5
tsvc.2.s275	704.0	720.8	719.9	545.7	476.2	163.9
tsvc.2.vpvtv	88.3	87.3	89.5	89.6	87.1	87.1
tsvc.2.vtvtv	96.4	94.9	96.3	95.8	94.8	94.8
<i>Reduction</i>						
argmax_with_index	71.6	71.5	71.5	71.7	178.6	71.7
quasi.affine.reduce_odd	63.8	66.3	63.6	63.6	67.4	63.5
scan.affine.decay	118.2	—	118.3	148.0	147.5	235.3
tsvc.2.s311	135.5	135.7	135.5	135.4	135.4	76.2
tsvc.2.s3110	120.2	120.8	120.4	121.5	315.3	121.9
tsvc.2.s3111	126.5	126.3	124.9	728.6	314.0	125.8
tsvc.2.s3112	79.8	79.8	79.9	79.8	79.7	79.9
tsvc.2.s316	275.8	276.0	80.0	80.2	123.1	69.3
tsvc.2.s318	80.2	80.2	79.7	80.0	290.7	252.6
tsvc.2.s319	75.0	74.9	74.5	74.8	76.6	74.1
<i>Recurrence</i>						
tsvc.2.s323	83.0	82.8	88.2	80.1	82.0	82.7
<i>Scalar Exp.</i>						
tsvc.2.s252	82.3	85.9	87.7	85.6	80.7	83.9
tsvc.2.s255	77.6	77.5	77.3	77.7	80.6	66.1
<i>Linear Dep.</i>						
tsvc.2.s115	83.1 [†]	84.0 [†]	83.3 [†]	84.3 [†]	141.0	86.8 [†]
tsvc.2.s119	70.1	70.5	70.4	70.0	69.1	56.9
<i>Dep. Dist.</i>						
ext_war_unit	61.6	61.3	61.5	60.4	61.5	61.5
versioned_distance_update	302.9	—	300.6	330.8	325.2	123.1
<i>Node Split.</i>						
tsvc.2.s1244	53.9	53.1	53.6	53.6	53.5	58.0
<i>Interproc.</i>						
tsvc.2.s152	63.2	51.5	62.5	62.6	63.1	51.6
<i>Indirect</i>						
scatter_accum_dup	351.2	—	352.4	371.4	409.8	938.4
segment_reduce_ragged	39.0	—	38.9	39.8	56.1	38.0
tsvc.2.s4112	465.5	463.7	463.0	486.2	468.9	449.7
tsvc.2.vag	447.8	439.4	449.6	464.6	438.2	482.5
<i>Packing</i>						
compact_threshold_pack	110.9	—	111.4	111.1	106.2	143.7
<i>Wavefront</i>						
wf.diff_skew [†]	59.9	60.4	60.1	60.1	61.5	66.3
wf.triangular [†]	99.5	100.5	99.7	99.5	382.8	153.4

Table 1: The loop-transformation matrix: minimum of 30 runs in milliseconds, preset M, float64, one core of a GH200 node. Bold: fastest valid version. —: version does not exist. ‡: output does not match the NumPy oracle; the time is shown but not used. †: passes validation only vacuously at this size (Section 6.2).

Extracted Kernel Catalogue

Skill Texts

Reproducibility



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

- ☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

First name(s):

With my signature I confirm the following:

- I have adhered to the rules set out in the [Citation Guidelines](#).
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

Signature(s)

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ For further information please consult the ETH Zurich websites, e.g. <https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html> and <https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html> (subject to change).